

MATEMÁTICA

UNIDAD N°1

TRABAJO INTEGRADOR

1ERA PARTE

MATEMÁTICA

TRABAJO PRÁCTICO INTEGRADOR - UNIDAD 1

Lógica Computacional: Validando algoritmos con R

1. El Salto de la Teoría al Código

Hasta ahora, hemos resuelto la lógica de forma manual. En el mundo de Sistemas, la lógica es el motor de los programas. R y RStudio son herramientas profesionales que nos permiten manejar valores de verdad de forma masiva. En R, los valores **Verdadero** y **Falso** se representan como `TRUE` y `FALSE` (siempre en mayúsculas).

2. Equivalencias Críticas (Tu "Diccionario" de Supervivencia)

Para programar los desafíos, debés traducir los símbolos lógicos a operadores de código:

- **Conjunción (y):** Se usa el símbolo `&`. Solo devuelve `TRUE` si ambos lados son ciertos.
- **Disyunción (o):** Se usa la barra vertical `|`. Devuelve `TRUE` si al menos uno es cierto.
- **Negación (no):** Se usa el signo de exclamación `!`. Invierte el valor (lo que es `TRUE` pasa a ser `FALSE`).
- **Condiciona (entonces):** R no tiene un símbolo "flecha". Usamos la ley de equivalencia: **no p o q**, que en código se escribe: `!p | q`.

Guía resumen de Equivalencias Lógicas en R

Concepto Lógico	Operador en R	Ejemplo de uso
Conjunción (y)	<code>&</code>	<code>p & q</code>
Disyunción (o)	<code> </code>	<code>p q</code>
Negación (no)	<code>!</code>	<code>!p</code>
Condiciona (entonces)	<code>!p q</code>	(Equivalencia lógica)
Bicondiciona (si y solo si)	<code>p == q</code>	(Comparación de valores)
Cuantificador Universal	<code>all()</code>	<code>all(vector)</code>
Cuantificador Existencial	<code>any()</code>	<code>any(vector)</code>

3. Los Tres Desafíos Integradores

Desafío 1: Automatización de Tablas de Verdad

Contexto: En lugar de dibujar la tabla, vamos a crear una base de datos pequeña que contenga todas las posibilidades. **Instrucciones:**

1. Creamos los vectores de las variables:

```
o p <- c(TRUE, TRUE, FALSE, FALSE)
```

MATEMÁTICA

- `q <- c(TRUE, FALSE, TRUE, FALSE)`
- 2. Creamos la tabla: `tabla <- data.frame(p, q)`
- 3. Tu tarea es agregar una columna llamada **Resultado** que calcule una operación compleja, por ejemplo: `!(p & q)`.
- 4. Al ejecutar `print(tabla)`, R te mostrará la tabla de verdad completa calculada en un milisegundo.

Desafío 2: Motor de Decisiones (Caso: Ingreso a Sistemas)

Contexto: Un programa debe decidir si permite el acceso a un servidor crítico. **La Regla de Negocio:** "Se permite el ingreso si el usuario tiene la Clave correcta **Y** (es Administrador **O** tiene autorización especial)". **Instrucciones:**

1. Definí tres variables: `clave`, `admin`, `autorizacion` y asignales valores `TRUE` o `FALSE`.
2. Escribí la fórmula lógica usando paréntesis para agrupar: `acceso <- clave & (admin | autorizacion)`.
3. Usá la función `print(acceso)` para ver si el motor funciona.
4. **Prueba obligatoria:** Cambiá los valores de las variables para simular un usuario que tiene la clave pero no es admin ni tiene autorización. El sistema debería devolver `FALSE`.

Desafío 3: Lógica de Predicados y Big Data

Contexto: Trabajamos con un "universo" de datos. Usamos funciones que actúan como cuantificadores. **Instrucciones:**

1. Creamos un vector con el estado de una flota de 100 servidores: `estado_servidores <- c(rep(TRUE, 95), rep(FALSE, 5))` (Esto crea 95 activos y 5 caídos).
2. **Cuantificador Universal (Todos):** Usamos la función `all(estado_servidores)`. ¿Están TODOS activos?
3. **Cuantificador Existencial (Existe):** Usamos la función `any(!estado_servidores)`. ¿Existe alguno que NO esté activo?
4. Tu misión es explicar por qué el resultado de `all()` cambia si uno de los servidores falla.

4. ¿Qué se evalúa? (Rúbrica de éxito)

- **Funcionalidad:** Que el script corra sin errores en RStudio.
- **Comentarios:** Que cada línea importante tenga un comentario explicando la lógica detrás (Ejemplo: `# Aplicamos ley de Morgan aquí`).
- **Interpretación:** Tu capacidad para explicar en el mismo script si el resultado de una operación es una Tautología o una Contingencia.